

## model/layers.py (Q&A Documentation)

**Q: What does this file implement at a high level?** **A:** This file defines reusable neural building blocks (embeddings, FM interaction, MLP variants, cross/AFM layers) shared across multiple models.

**Q: Why keep these layers in a separate file?** **A:** It avoids duplication across models and keeps implementations consistent (e.g., embedding offsets, FM math, activation/dropout choices).

**Q: What does this file export (public API)?** **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
class FeaturesLinear # methods: __init__, forward
class FeaturesEmbedding # methods: __init__, forward
class SeqFeatureEmbedding # methods: __init__, forward
class FactorizationMachine # methods: __init__, forward
class MultiLayerPerceptron # methods: __init__, forward
class DurationMultiLayerPerceptron # methods: __init__, forward
class Swish # methods: forward
class MultiLayerPerceptronD2Q # methods: __init__, forward
class MultiLayerPerceptronTPM # methods: __init__, forward
class AttentionalFactorizationMachine # methods: __init__, forward
class CrossNetwork # methods: __init__, forward
```

**Q: What are the main dependencies (imports) and why do they matter?** **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import numpy as np
import torch
import torch.nn.functional as F
```

```
class FeaturesLinear(torch.nn.Module):
```

```
    def __init__(self, field_dims, output_dim=1):
        super().__init__()
        self.fc = torch.nn.Embedding(sum(field_dims), output_dim)
        self.bias = torch.nn.Parameter(torch.zeros((output_dim,)))
        self.offsets = np.array((0, *np.cumsum(field_dims)[: -1]), dtype=np.long)

    def forward(self, x):
        x = x + x.new_tensor(self.offsets).unsqueeze(0)
        return torch.sum(self.fc(x), dim=1) + self.bias
```

```

class FeaturesEmbedding(torch.nn.Module):

    def __init__(self, field_dims, embed_dim):
        """
        :param x: Long tensor of size ``(batch_size, num_fields)``
        """
        super().__init__()
        self.embedding = torch.nn.Embedding(sum(field_dims), embed_dim)
        self.offsets = np.array((0, *np.cumsum(field_dims[:-1])), dtype=np.long)
        torch.nn.init.xavier_uniform_(self.embedding.weight.data)

    def forward(self, x):
        x = x + x.new_tensor(self.offsets).unsqueeze(0)
        return self.embedding(x)

class SeqFeatureEmbedding(torch.nn.Module):

    def __init__(self, vocab_sizes, embed_dim):
        super().__init__()
        self.embedding = torch.nn.Embedding(sum(vocab_sizes), embed_dim)
        self.offsets = np.array((0, *np.cumsum(vocab_sizes[:-1])), dtype=np.long)
        torch.nn.init.xavier_uniform_(self.embedding.weight.data)

    def forward(self, x_list):
        embs = list()
        for i, x in enumerate(x_list):
            embs.append(self.embedding(x + self.offsets[i]).sum(dim=1, keepdims=True))
        emb = torch.concat(embs, dim=1)
        return emb

class FactorizationMachine(torch.nn.Module):

```

**Q: What is FeaturesLinear and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on FeaturesLinear?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```

__init__
forward

```

**Q: What does the implementation look like for FeaturesLinear?** **A:** Excerpt from the file:

```

class FeaturesLinear(torch.nn.Module):

    def __init__(self, field_dims, output_dim=1):
        super().__init__()
        self.fc = torch.nn.Embedding(sum(field_dims), output_dim)

```

```

        self.bias = torch.nn.Parameter(torch.zeros((output_dim,)))
        self.offsets = np.array((0, *np.cumsum(field_dims)[: -1]), dtype=np.long)

    def forward(self, x):
        x = x + x.new_tensor(self.offsets).unsqueeze(0)
        return torch.sum(self.fc(x), dim=1) + self.bias

```

**Q: Why is FeaturesLinear needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

**Q: What is FeaturesEmbedding and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on FeaturesEmbedding?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```

__init__
forward

```

**Q: What does the implementation look like for FeaturesEmbedding?** **A:** Excerpt from the file:

```

class FeaturesEmbedding(torch.nn.Module):

    def __init__(self, field_dims, embed_dim):
        """
        :param x: Long tensor of size ``(batch_size, num_fields)``
        """
        super().__init__()
        self.embedding = torch.nn.Embedding(sum(field_dims), embed_dim)
        self.offsets = np.array((0, *np.cumsum(field_dims)[: -1]), dtype=np.long)
        torch.nn.init.xavier_uniform_(self.embedding.weight.data)

    def forward(self, x):
        x = x + x.new_tensor(self.offsets).unsqueeze(0)
        return self.embedding(x)

```

**Q: Why is FeaturesEmbedding needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

**Q: What is SeqFeatureEmbedding and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on SeqFeatureEmbedding?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```

__init__
forward

```

**Q: What does the implementation look like for SeqFeatureEmbedding?** **A:** Excerpt from the file:

```
class SeqFeatureEmbedding(torch.nn.Module):

    def __init__(self, vocab_sizes, embed_dim):
        super().__init__()
        self.embedding = torch.nn.Embedding(sum(vocab_sizes), embed_dim)
        self.offsets = np.array((0, *np.cumsum(vocab_sizes[:-1])), dtype=np.long)
        torch.nn.init.xavier_uniform_(self.embedding.weight.data)

    def forward(self, x_list):
        embs = list()
        for i, x in enumerate(x_list):
            embs.append(self.embedding(x + self.offsets[i]).sum(dim=1, keepdims=True))
        emb = torch.concat(embs, dim=1)
        return emb
```

**Q: Why is SeqFeatureEmbedding needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

**Q: What is FactorizationMachine and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on FactorizationMachine?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```
__init__
forward
```

**Q: What does the implementation look like for FactorizationMachine?** **A:** Excerpt from the file:

```
class FactorizationMachine(torch.nn.Module):

    def __init__(self, reduce_sum=True):
        super().__init__()
        self.reduce_sum = reduce_sum

    def forward(self, x):
        """
        :param x: Float tensor of size ``(batch_size, num_fields, embed_dim)``
        """
        square_of_sum = torch.sum(x, dim=1) ** 2
        sum_of_square = torch.sum(x ** 2, dim=1)
        ix = square_of_sum - sum_of_square
        if self.reduce_sum:
            ix = torch.sum(ix, dim=1, keepdim=True)
        return 0.5 * ix
```

**Q: Why is FactorizationMachine needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

**Q: What is MultiLayerPerceptron and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on MultiLayerPerceptron?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```
__init__  
forward
```

**Q: What does the implementation look like for MultiLayerPerceptron?** **A:** Excerpt from the file:

```
class MultiLayerPerceptron(torch.nn.Module):  
  
    def __init__(self, input_dim, embed_dims, dropout, output_layer=True):  
        super().__init__()   
        layers = list()  
        for embed_dim in embed_dims:  
            layers.append(torch.nn.Linear(input_dim, embed_dim))  
            layers.append(torch.nn.BatchNorm1d(embed_dim))  
            layers.append(torch.nn.ReLU())  
            layers.append(torch.nn.Dropout(p=dropout))  
            input_dim = embed_dim  
        if output_layer:  
            layers.append(torch.nn.Linear(input_dim, 1))  
        self.mlp = torch.nn.Sequential(*layers)  
  
    def forward(self, x):  
        """  
        :param x: Float tensor of size ``(batch_size, embed_dim)``  
        """  
        return self.mlp(x)
```

**Q: Why is MultiLayerPerceptron needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

**Q: What is DurationMultiLayerPerceptron and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on DurationMultiLayerPerceptron?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```
__init__  
forward
```

**Q: What does the implementation look like for DurationMultiLayerPerceptron?** **A:** Excerpt from the file:

```

class DurationMultiLayerPerceptron(torch.nn.Module):

    def __init__(self, input_dim, embed_dims, dropout, output_layer=True):
        super().__init__()
        self.mlps = torch.nn.ModuleList()
        for embed_dim in embed_dims:
            layers = list()
            layers.append(torch.nn.Linear(input_dim + 1, embed_dim))
            layers.append(torch.nn.BatchNorm1d(embed_dim))
            layers.append(torch.nn.ReLU())
            layers.append(torch.nn.Dropout(p=dropout))
            self.mlps.append(torch.nn.Sequential(*layers))
            input_dim = embed_dim

    def forward(self, x, duration):
        """
        :param x: Float tensor of size ``(batch_size, embed_dim)``
        """
        for mlp in self.mlps:
            x = mlp(torch.concat([x, duration], dim=1))
        return x

```

**Q: Why is DurationMultiLayerPerceptron needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

**Q: What is Swish and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on Swish?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

forward

**Q: What does the implementation look like for Swish?** **A:** Excerpt from the file:

```

class Swish(torch.nn.Module):
    def forward(self, x):
        return x * torch.sigmoid(x)

```

**Q: Why is Swish needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

**Q: What is MultiLayerPerceptronD2Q and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on MultiLayerPerceptronD2Q?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

\_\_init\_\_  
forward

**Q: What does the implementation look like for MultiLayerPerceptronD2Q?** **A:** Excerpt from the file:

```
class MultiLayerPerceptronD2Q(torch.nn.Module):

    def __init__(self, input_dim, embed_dims, dropout, output_layer=True):
        super().__init__()
        layers = list()
        for embed_dim in embed_dims:
            layers.append(torch.nn.Linear(input_dim, embed_dim))
            layers.append(torch.nn.BatchNorm1d(embed_dim))
            layers.append(Swish())
            layers.append(torch.nn.Dropout(p=dropout))
            input_dim = embed_dim
        if output_layer:
            layers.append(torch.nn.Linear(input_dim, 1))
        self.mlp = torch.nn.Sequential(*layers)

    def forward(self, x):
        """
        :param x: Float tensor of size ``(batch_size, embed_dim)``
        """
        return self.mlp(x)
```

**Q: Why is MultiLayerPerceptronD2Q needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

**Q: What is MultiLayerPerceptronTPM and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on MultiLayerPerceptronTPM?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```
__init__
forward
```

**Q: What does the implementation look like for MultiLayerPerceptronTPM?** **A:** Excerpt from the file:

```
class MultiLayerPerceptronTPM(torch.nn.Module):

    def __init__(self, input_dim, embed_dims, dropout, class_num, output_layer=True):
        super().__init__()
        layers = list()
        for embed_dim in embed_dims:
            layers.append(torch.nn.Linear(input_dim, embed_dim))
            layers.append(torch.nn.BatchNorm1d(embed_dim))
            layers.append(Swish())
            layers.append(torch.nn.Dropout(p=dropout))
            input_dim = embed_dim
        if output_layer:
            layers.append(torch.nn.Linear(input_dim, class_num))
        self.mlp = torch.nn.Sequential(*layers)

    def forward(self, x):
        """
```

```

:param x: Float tensor of size ``(batch_size, embed_dim)``
"""
return self.mlp(x)

```

**Q: Why is MultiLayerPerceptronTPM needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

**Q: What is AttentionalFactorizationMachine and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on AttentionalFactorizationMachine?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```

__init__
forward

```

**Q: What does the implementation look like for AttentionalFactorizationMachine?** **A:** Excerpt from the file:

```

class AttentionalFactorizationMachine(torch.nn.Module):

    def __init__(self, embed_dim, attn_size, dropouts):
        super().__init__()
        self.attention = torch.nn.Linear(embed_dim, attn_size)
        self.projection = torch.nn.Linear(attn_size, 1)
        self.fc = torch.nn.Linear(embed_dim, 1)
        self.dropouts = dropouts

    def forward(self, x):
        """
        :param x: Float tensor of size ``(batch_size, num_fields, embed_dim)``
        """
        num_fields = x.shape[1]
        row, col = list(), list()
        for i in range(num_fields - 1):
            for j in range(i + 1, num_fields):
                row.append(i), col.append(j)
        p, q = x[:, row], x[:, col]
        inner_product = p * q
        attn_scores = F.relu(self.attention(inner_product))
        attn_scores = F.softmax(self.projection(attn_scores), dim=1)
        attn_scores = F.dropout(attn_scores, p=self.dropouts[0], training=self.training)
        attn_output = torch.sum(attn_scores * inner_product, dim=1)
        attn_output = F.dropout(attn_output, p=self.dropouts[1], training=self.training)
        return self.fc(attn_output)

```

**Q: Why is AttentionalFactorizationMachine needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.



**Q: What is CrossNetwork and what responsibility does it have?** **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

**Q: What are the important methods on CrossNetwork?** **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```
__init__
forward
```

**Q: What does the implementation look like for CrossNetwork?** **A:** Excerpt from the file:

```
class CrossNetwork(torch.nn.Module):

    def __init__(self, input_dim, num_layers):
        super().__init__()
        self.num_layers = num_layers
        self.w = torch.nn.ModuleList([
            torch.nn.Linear(input_dim, 1, bias=False) for _ in range(num_layers)
        ])
        self.b = torch.nn.ParameterList([
            torch.nn.Parameter(torch.zeros((input_dim,))) for _ in range(num_layers)
        ])

    def forward(self, x):
        """
        :param x: Float tensor of size ``(batch_size, num_fields, embed_dim)``
        """
        x0 = x
        for i in range(self.num_layers):
            xw = self.w[i](x)
            x = x0 * xw + self.b[i] + x
        return x
```

**Q: Why is CrossNetwork needed by training scripts?** **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

**Q: Example: end-to-end data flow involving this file** **A:** Core pipeline shape (schematic):

```
for features, label in dataloaders["train"]:
    out = model(features)
    loss = ...
    loss.backward(); optimizer.step()
```

**Q: Full source code of the Python file** **A:** The complete file is included verbatim for reference.

```
import numpy as np
import torch
import torch.nn.functional as F

class FeaturesLinear(torch.nn.Module):

    def __init__(self, field_dims, output_dim=1):
        super().__init__()
        self.fc = torch.nn.Embedding(sum(field_dims), output_dim)
        self.bias = torch.nn.Parameter(torch.zeros((output_dim,)))
        self.offsets = np.array((0, *np.cumsum(field_dims)[-1:]), dtype=np.long)
```

```

def forward(self, x):
    x = x + x.new_tensor(self.offsets).unsqueeze(0)
    return torch.sum(self.fc(x), dim=1) + self.bias

class FeaturesEmbedding(torch.nn.Module):

    def __init__(self, field_dims, embed_dim):
        """
        :param x: Long tensor of size ``(batch_size, num_fields)``
        """
        super().__init__()
        self.embedding = torch.nn.Embedding(sum(field_dims), embed_dim)
        self.offsets = np.array((0, *np.cumsum(field_dims)[-1]), dtype=np.long)
        torch.nn.init.xavier_uniform_(self.embedding.weight.data)

    def forward(self, x):
        x = x + x.new_tensor(self.offsets).unsqueeze(0)
        return self.embedding(x)

class SeqFeatureEmbedding(torch.nn.Module):

    def __init__(self, vocab_sizes, embed_dim):
        super().__init__()
        self.embedding = torch.nn.Embedding(sum(vocab_sizes), embed_dim)
        self.offsets = np.array((0, *np.cumsum(vocab_sizes)[-1]), dtype=np.long)
        torch.nn.init.xavier_uniform_(self.embedding.weight.data)

    def forward(self, x_list):
        embs = list()
        for i, x in enumerate(x_list):
            embs.append(self.embedding(x + self.offsets[i]).sum(dim=1, keepdims=True))
        emb = torch.concat(embs, dim=1)
        return emb

class FactorizationMachine(torch.nn.Module):

    def __init__(self, reduce_sum=True):
        super().__init__()
        self.reduce_sum = reduce_sum

    def forward(self, x):
        """
        :param x: Float tensor of size ``(batch_size, num_fields, embed_dim)``
        """
        square_of_sum = torch.sum(x, dim=1) ** 2
        sum_of_square = torch.sum(x ** 2, dim=1)
        ix = square_of_sum - sum_of_square
        if self.reduce_sum:
            ix = torch.sum(ix, dim=1, keepdim=True)
        return 0.5 * ix

class MultiLayerPerceptron(torch.nn.Module):

    def __init__(self, input_dim, embed_dims, dropout, output_layer=True):
        super().__init__()
        layers = list()
        for embed_dim in embed_dims:
            layers.append(torch.nn.Linear(input_dim, embed_dim))
            layers.append(torch.nn.BatchNorm1d(embed_dim))
            layers.append(torch.nn.ReLU())
            layers.append(torch.nn.Dropout(p=dropout))
            input_dim = embed_dim
        if output_layer:

```

```

        layers.append(torch.nn.Linear(input_dim, 1))
        self.mlp = torch.nn.Sequential(*layers)

    def forward(self, x):
        """
        :param x: Float tensor of size ``(batch_size, embed_dim)``
        """
        return self.mlp(x)

class DurationMultiLayerPerceptron(torch.nn.Module):

    def __init__(self, input_dim, embed_dims, dropout, output_layer=True):
        super().__init__()
        self.mlps = torch.nn.ModuleList()
        for embed_dim in embed_dims:
            layers = list()
            layers.append(torch.nn.Linear(input_dim + 1, embed_dim))
            layers.append(torch.nn.BatchNorm1d(embed_dim))
            layers.append(torch.nn.ReLU())
            layers.append(torch.nn.Dropout(p=dropout))
            self.mlps.append(torch.nn.Sequential(*layers))
            input_dim = embed_dim

    def forward(self, x, duration):
        """
        :param x: Float tensor of size ``(batch_size, embed_dim)``
        """
        for mlp in self.mlps:
            x = mlp(torch.concat([x, duration], dim=1))
        return x

class Swish(torch.nn.Module):
    def forward(self, x):
        return x * torch.sigmoid(x)

class MultiLayerPerceptronD2Q(torch.nn.Module):

    def __init__(self, input_dim, embed_dims, dropout, output_layer=True):
        super().__init__()
        layers = list()
        for embed_dim in embed_dims:
            layers.append(torch.nn.Linear(input_dim, embed_dim))
            layers.append(torch.nn.BatchNorm1d(embed_dim))
            layers.append(Swish())
            layers.append(torch.nn.Dropout(p=dropout))
            input_dim = embed_dim
        if output_layer:
            layers.append(torch.nn.Linear(input_dim, 1))
        self.mlp = torch.nn.Sequential(*layers)

    def forward(self, x):
        """
        :param x: Float tensor of size ``(batch_size, embed_dim)``
        """
        return self.mlp(x)

class MultiLayerPerceptronTPM(torch.nn.Module):

    def __init__(self, input_dim, embed_dims, dropout, class_num, output_layer=True):
        super().__init__()
        layers = list()
        for embed_dim in embed_dims:
            layers.append(torch.nn.Linear(input_dim, embed_dim))
            layers.append(torch.nn.BatchNorm1d(embed_dim))
            layers.append(Swish())

```

```

        layers.append(torch.nn.Dropout(p=dropout))
        input_dim = embed_dim
    if output_layer:
        layers.append(torch.nn.Linear(input_dim, class_num))
    self.mlp = torch.nn.Sequential(*layers)

def forward(self, x):
    """
    :param x: Float tensor of size ``(batch_size, embed_dim)``
    """
    return self.mlp(x)

class AttentionalFactorizationMachine(torch.nn.Module):

    def __init__(self, embed_dim, attn_size, dropouts):
        super().__init__()
        self.attention = torch.nn.Linear(embed_dim, attn_size)
        self.projection = torch.nn.Linear(attn_size, 1)
        self.fc = torch.nn.Linear(embed_dim, 1)
        self.dropouts = dropouts

    def forward(self, x):
        """
        :param x: Float tensor of size ``(batch_size, num_fields, embed_dim)``
        """
        num_fields = x.shape[1]
        row, col = list(), list()
        for i in range(num_fields - 1):
            for j in range(i + 1, num_fields):
                row.append(i), col.append(j)
        p, q = x[:, row], x[:, col]
        inner_product = p * q
        attn_scores = F.relu(self.attention(inner_product))
        attn_scores = F.softmax(self.projection(attn_scores), dim=1)
        attn_scores = F.dropout(attn_scores, p=self.dropouts[0], training=self.training)
        attn_output = torch.sum(attn_scores * inner_product, dim=1)
        attn_output = F.dropout(attn_output, p=self.dropouts[1], training=self.training)
        return self.fc(attn_output)

class CrossNetwork(torch.nn.Module):

    def __init__(self, input_dim, num_layers):
        super().__init__()
        self.num_layers = num_layers
        self.w = torch.nn.ModuleList([
            torch.nn.Linear(input_dim, 1, bias=False) for _ in range(num_layers)
        ])
        self.b = torch.nn.ParameterList([
            torch.nn.Parameter(torch.zeros((input_dim,))) for _ in range(num_layers)
        ])

    def forward(self, x):
        """
        :param x: Float tensor of size ``(batch_size, num_fields, embed_dim)``
        """
        x0 = x
        for i in range(self.num_layers):
            xw = self.w[i](x)
            x = x0 * xw + self.b[i] + x
        return x

```